Introducing Arduino

MEGN 441

COLORADO SCHOOL OF MINES

Robot of the Week

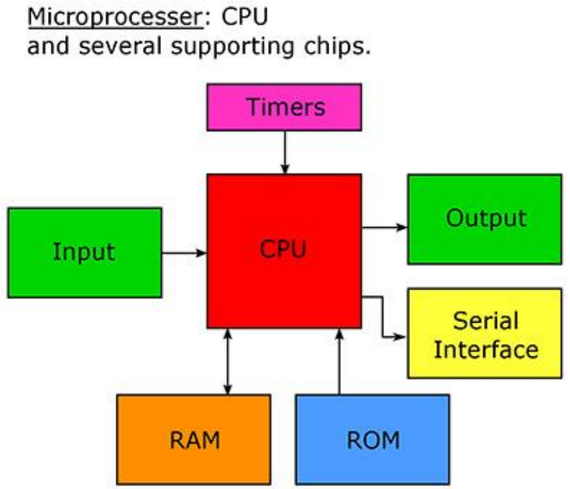
- In-class presentation with lab partners
- 10 minute talk (+2 minutes for questions)
- See Canvas for rubric + guidelines (coming soon)
- Guidelines
  - Robots need to be “real” systems (or simulations of hardware in prototype phase)
  - Describe its functions, features, and relevance

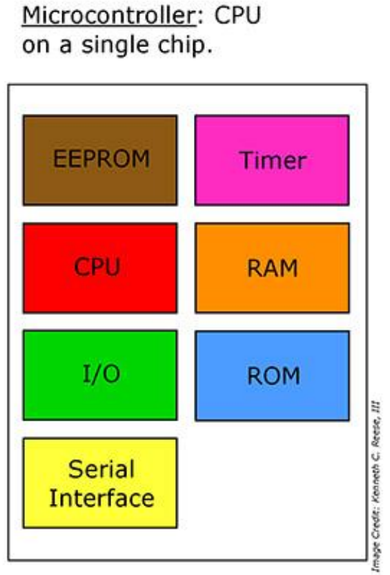
COLORADO SCHOOL OF MINES

Microprocessor vs Microcontroller

Microprocessors

- Just contain the CPU
- Additional hardware needed to function must be added by the user
- Can typically handle more intense processing





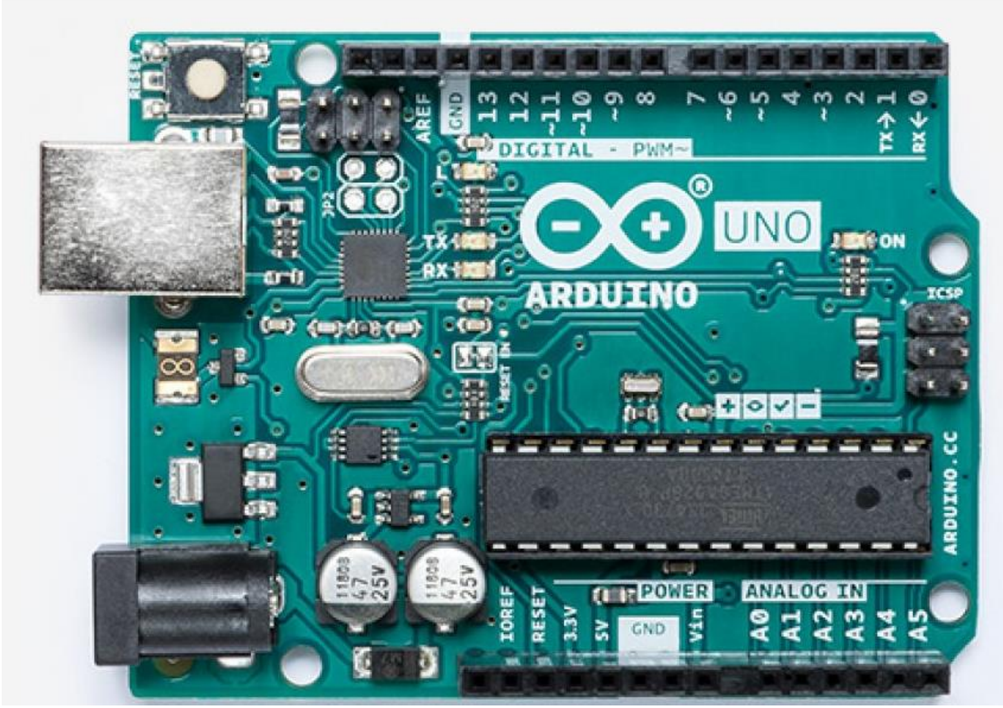
Microcontrollers

- Contain all of the hardware needed to function (memory, processor, peripherals)
- Able to operate as a single chip

COLORADO SCHOOL OF MINES

Arduino Uno specifications

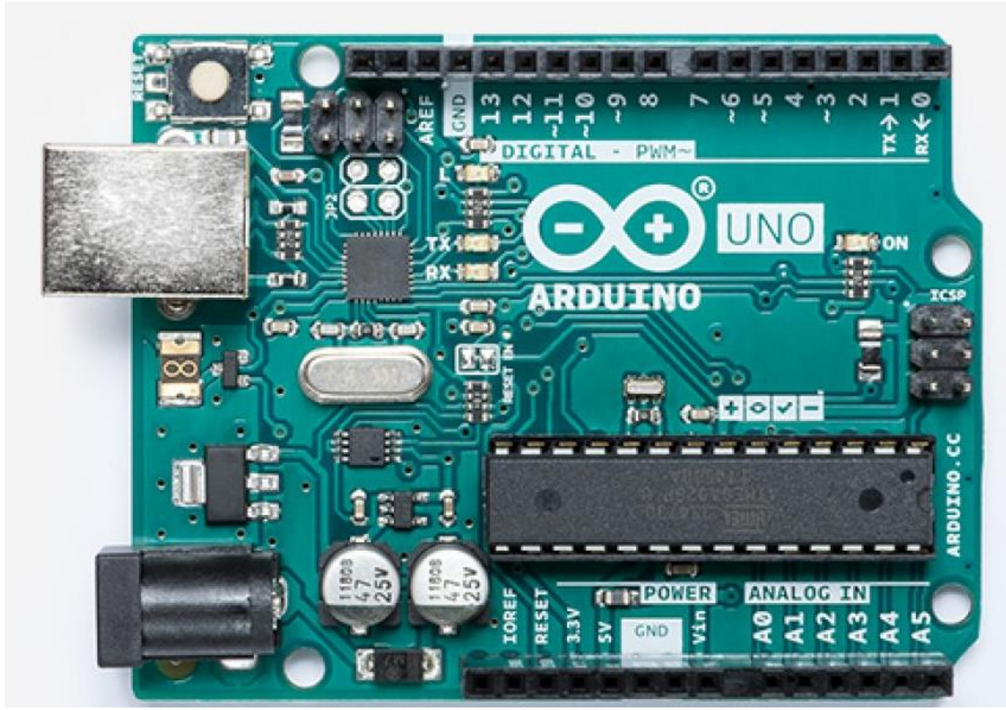
- Uses ATmega328p microcontroller
- Processor: 8-bit
- Clock Speed: 16MHz
- Memory:
  - 32KB Flash
    - 0.5KB bootloader
    - Remainder for user programs
  - 2KB SRAM
    - Dynamic variables
  - 1KB EEPROM
    - Can store variables that need to persist (non-volatile)



COLORADO SCHOOL OF MINES

Arduino Uno specifications

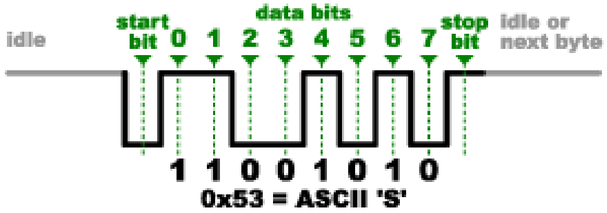
- Pins
  - 14 Digital I/O pins
  - 6 PWM pins (operate at 490 Hz)
  - 6 Analog inputs (10-bit resolution, range 0-1023)
    - Can also act as digital I/O pins
- Pin Current
  - I/O pins: 40mA (20mA is recommended max)
  - Vcc/GND: 200mA
- Power
  - 7-12V recommended (6-20V maximum)



COLORADO SCHOOL OF MINES

Serial communication

- D0 (RX), D1 (TX) used for UART (Universal Asynchronous Receiver/Transmitter)
  - Needs to be attached to opposite pin on other board (TX -> RX and RX -> TX)
  - Other pins can be used by importing the “SoftwareSerial.h” library. This is software based rather than hardware based, so it has limitations.
  - https://learn.sparkfun.com/tutorials/serial-communication/all
- A4 (SDA), A5 (SCL) used for I2C (Inter-Integrated Circuit)
  - https://learn.sparkfun.com/tutorials/i2c/all
- D11 (MOSI), D12 (MISO), D13 (SCK), D10 (SS) are used for SPI (Serial Peripheral Interface)
  - https://learn.sparkfun.com/tutorials/serial-peripheral-interface-spi/all

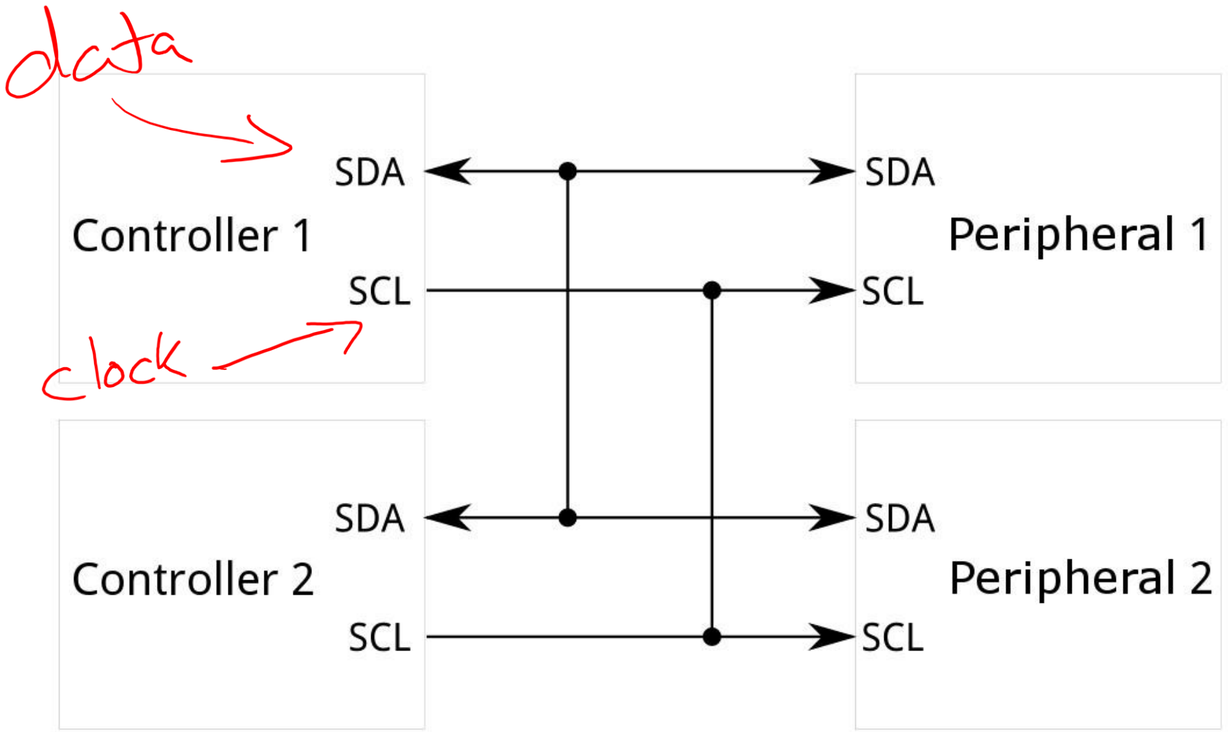


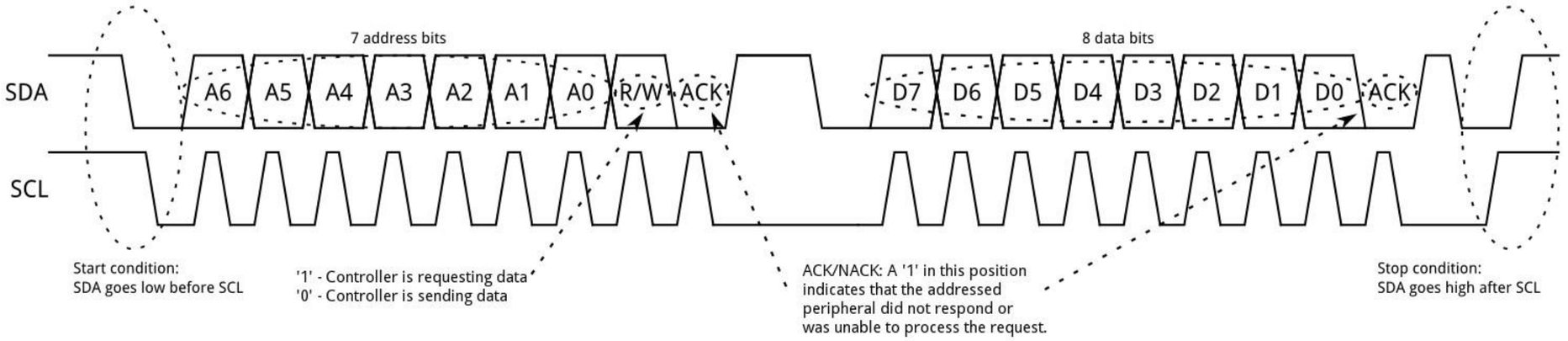
COLORADO SCHOOL OF MINES

Serial communication

A4 (SDA), A5 (SCL) used for I2C (Inter-Integrated Circuit)

- https://learn.sparkfun.com/tutorials/i2c/all

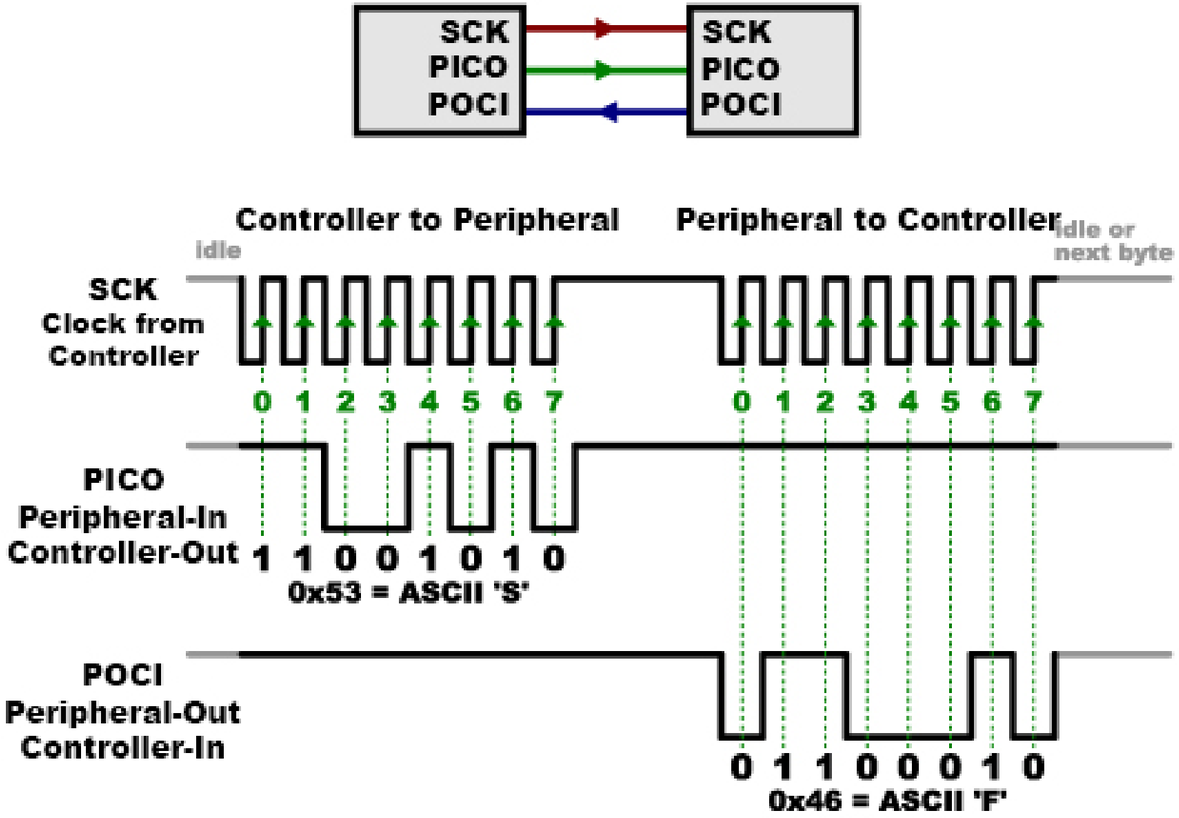




COLORADO SCHOOL OF MINES

Serial communication

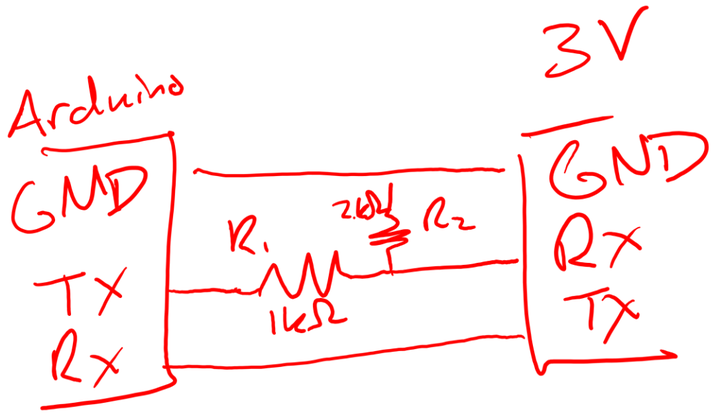
- D11 (MOSI), D12 (MISO), D13 (SCK), D10 (SS) are used for SPI (Serial Peripheral Interface)
  - https://learn.sparkfun.com/tutorials/serial-peripheral-interface-spi/all



COLORADO SCHOOL OF MINES

Logical output considerations

- Arduino operates on 5V logic
- HIGH and LOW read as follows:
  - LOW: <= 0.3*Vcc (for 5V that's 1.5V)
  - HIGH: >= 0.6*Vcc (for 5V that's 3V)
  - Anything in between will have unknown behavior
- If communicating with 3V board
  - Can use a voltage divider for Arduino output



$$V_{out} = \frac{R_2}{R_1 + R_2} V_{in}$$

$$= \frac{2}{3} 5V = 3.3V$$

COLORADO SCHOOL OF MINES

Logical output considerations

- PWM Outputs
  - Using as an analog output assumes the load acts as a low-pass filter
    - Multimeters act this way and will read a constant voltage
    - Oscilloscopes read at high frequencies so you will see the square wave pulses
  - If load does not act as low-pass filter (2 simple options)
    - Get a DAC board that outputs true analog
    - Add your own low-pass filter using RC circuits
      - Characteristic frequency: $f = \frac{1}{2\pi RC}$

Programming with Arduino

MEGN 441

COLORADO SCHOOL OF MINES

An Arduino Sketch

#include <package.h>

#define CONSTANT 2

Replaces all instances of CONSTANT with the value 2

setup()

Runs only once, before the loop()

loop()

Runs continuously, like a “while true” loop

Data types

```
// Both of these are okay.
int myVariable = 7;
unsigned int otherVariable;
// Once declared, variables can be changed.
myVariable = 8;
otherVariable = 9;
```

Keyword	Type	Memory Size	Range
int	Integer	2 bytes (16 bits)	-32,768 to 32,767
long	Long integer	4 bytes (32 bits)	-2,147,483,648 to 2,147,483,647
unsigned int	Unsigned integer	2 bytes	0 to 65535
unsigned long	Unsigned long	4 bytes	0 to 4,294,967,295
float	Floating point value (7 digits of precision)	4 bytes	-3.4028235E+38 to 3.4028235E+38
char	Character	1 byte (8 bits)	-128 to 127 representing ASCII code
bool	Boolean value	1 byte	0 or 1
byte	Single byte (unsigned)	1 byte	0 to 255

“const” modifier

- Creates a constant variable
- Values cannot be changed later in the code
- Saves on RAM because it can be stored with the program memory
- Can also use #define, but not encouraged

```
// Two constant variables.
const int BUTTON_PIN = 4;
#define MOTOR_PIN 7
// Don’t need the = when using #define.
```

Arrays

```
// Define arrays using brackets []
#define LEFT   1
#define RIGHT -1
int moveList[] = {100, LEFT, 50, RIGHT};
// All variables in the array are ints.
```

- Store multiple elements of a single type
- Use brackets with size inside ‘[size]’
- When initializing using curly braces, no size is needed
- Indexing starts at 0, ends at (size – 1)
- char[] needs extra element for NULL character ‘\0’
  - char my_word[9] = “sometext”;

Arithmetic operations

+	Add, unary plus
-	Subtract, unary minus
*	Multiply
/	Divide (truncates decimal values with “ints”)
%	Modulus, returns the remainder between “ints” (not valid for “floats”)
++	Increment by 1
--	Decrement by 1

Relational and logical operators

Return a 1 if the statement is TRUE, 0 if the statement is FALSE

==	Equal to
!=	Not equal to
>	Greater than
<	Less than
>=	Greater than or equal to
<=	Less than or equal to
&&	Logical AND
||	Logical OR
!	Logical NOT

Assignment + Operator

Can combine arithmetic operations and assignment for a variable

+=

-=

*=

/=

%=

```
int a = 0;
// These two statements are the same.
a = a + 5;
a += 5;
```

Control statements

- If/else
  - Statements are evaluated based on logical relationship.

```
if (condition) {
// Do this
} else {
// Do that
// Don't have to include else
}
```

- For
  - Sets a defined iterative loop
  - for (initial condition; loop criteria; statement to evaluate at the end of the loop)
  - *note the separation by semicolons ';'

```
for (int index=0; index < 5; index++)
{
// Do something
}
```

Control statements

- While
  - Iterative loop without specified incrementation
- Switch/case
  - Like a series of ‘if’ conditions
  - Remember to use “break” at the end of each case so that it exits and does not continue evaluating all of the cases below

```
while (condition)
{
// Do the thing
}
```

```
switch (value) {
   case 1:
      // Do things if value == 1;
      break;
   case 2:
      // Do things if value == 2;
      break;
   default:
      // Do things if no other condition applies
}
```

Quick tips

- Global variables must be defined outside of functions (not declared in the “setup()” function)
- Don’t put “if/else” statements outside of the functions
- Watch out for memory usage
  - If you get into the 90% range, you may start experiencing misbehavior
  - Happens when using ROS on Arduino, ROS packages are large and serial communication becomes disrupted
- Check your comparisons, make sure you use “==“ not “=“ in an if condition (“=“ will always return “true”)

Lab Today

- Build a device to lift a weight using a motor
- Get equipment from front table
- Create your own teams!
- Arduino code provided
- Output spreadsheet provided

What did we learn today?

COLORADO SCHOOL OF MINES